Medical Questions & Answers with Large Language Models

AI in Healthcare High-Risk Project

[redacted], The University of Texas at Austin, USA

1 INTRODUCTION

The main problem I wanted to explore was medical question and answering (Q&A) with large language models (LLMs).

Medical Q&A is challenging because it involves natural language processing (NLP) to understand the problem, medical domain knowledge expertise, and the ability to reason and generate an answer. Furthermore, knowledge in the medical domain is ever-growing, always changing, and spans across many languages, geographies, and sub-specialties. Mastering medical Q&A has many potential benefits and future use cases. For example, improving patient care through the use of clinical digital assistants.

My objectives for this project were:

- Explore public medical Q&A datasets.
- Experiment with few-shot prompt engineering for medical Q&A tasks.
- Benchmark and evaluate performance of various LLMs for medical Q&A.
- Fine-tune an open source LLM, to improve its performance on a medical Q&A task.

2 RELATED WORK

The paper that inspired this project tackled the problem of medical Q&A by developing domain-specific LLMs through continuous training and fine-tuning [3]. More specifically, the authors adapted Llama 2 base models to the Chinese medical domain. The main influence this paper had on my project was to make use of fine-tuning to improve the performance of LLama 2 base model on medical multiple-choice (English) Q&A tasks.

Another paper tackled the problem of medical Q&A by experimenting with various prompt engineering techniques, such as few-shot and ensemble methods [4]. The main influence this paper had on my project was to make use of few-shot prompt engineering.

3 METHODOLOGY

Figure 1 illustrates the various workflows used in this project. We deep-dive into the details in below subsections. However, at a high-level, the main workflows include:

(1) Process public MedMCQA (json) dataset to generate and store train & test prompts.
(2) Load train prompts and Llama 2 base model onto GPU, then fine-tune and save new model.
(3) Invoke target LLM with test prompts and store responses:
    - **GPT-3.5 Turbo** was deployed in Azure OpenAI service and invoked via REST API.
    - **Meta Llama** models (base & fine-tuned) were loaded onto a GPU and invoked locally.
(4) Given ground-truth labels and an LLM's test prompt responses, calculate evaluation metrics.

3.1 Q&A medical dataset

MedMCQA was the medical domain (multiple-choice) Q&A dataset used [**?**]. It is split roughly into 183k train, 4.2k development, and 6.2k test questions. However, because ground truth labels were purposefully excluded from the test set, I completely ignored the test set and treated the

Author's Contact Information: Diego M. Guerrero Cardenas, diego.guerrero@utexas.edu, The University of Texas at Austin, USA.

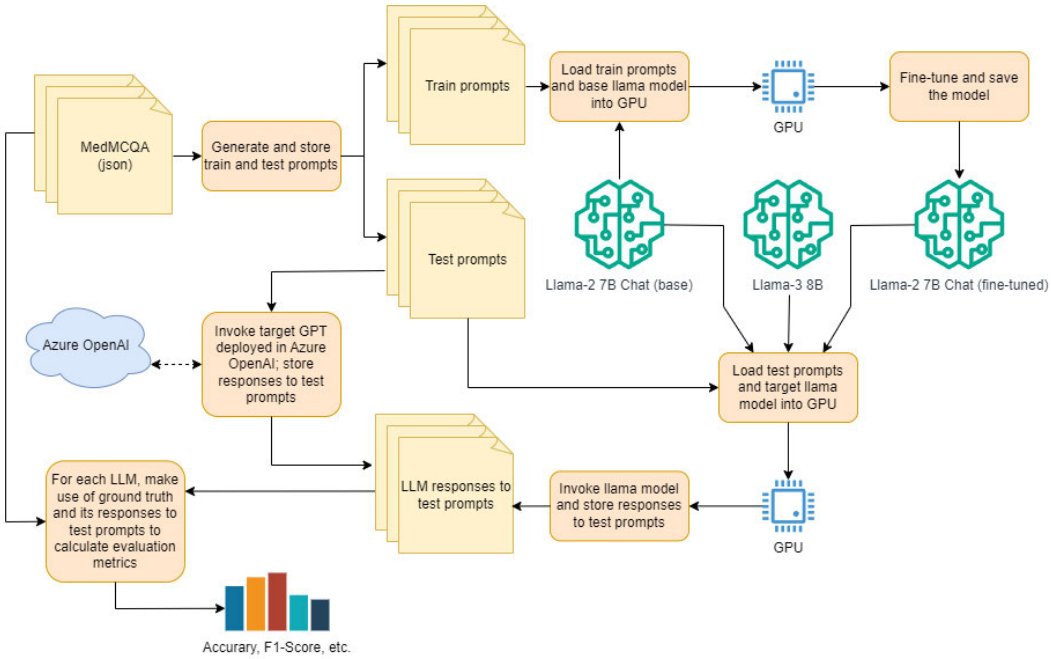

Fig. 1. High level workflows: Generate train and test prompts, fine-tune Llama-2 base model, invoke each LLM with test prompts, and evaluate LLM responses with ground truth.

development set as the new test set. Finally, each question is formatted as a JSON, it includes the main question's text, four answer choices with text for each, the question id, and the ground truth (correct answer choice).

3.2 Train & test prompt data generation

Each train and test prompt followed the same template and set of instructions, except the correct answer (to predict) was excluded from test prompts. Figure 2 shows an example of a train prompt. In the set of instructions, I told the LLM that it had the role of a medical professional and its task was to answer a single multiple-choice question, with *A*, *B*, *C*, or *D* as the answer. For few-shot learning, I gave the LLM 10 random Q&A pair examples.

Each question in MedMCQA train and development sets was transformed into a train and test prompt, respectively. Such transformation was via Python script and required no special compute.

3.3 Fine-tuning Llama-2 base model

Llama-2 is an open source large language model from Meta released in 2023 [6]. Multiple variants of this model exist. For the purpose of this project, Llama-2 chat (base) model with 7 billion parameters was fine-tuned for medical (multiple-choice) Q&A.

A requirement for fine-tuning Llama 2 was to do so on a GPU with at least 25GB of RAM. Thus, a single GPU from Google Colab Pro was used. Furthermore, to reduce memory footprint, train batch size was set to one [2].

Parameter efficient fine tuning (PEFT) with low-rank adaptation (LoRA) was used [5]. I fine-tuned Llama 2 (base) model for 5 epochs and with 1,000 random train prompts. End-2-end fine-tuning took approximately 5 hours to complete.

```
You are a medical professional taking a medical exam. Your task is to
answer one multiple choice question to the best of your abilities. Below
are 2 example questions and corresponding answers:

Example 1
Question:
Post extraction bleeding in a leukaemic patient is due to:
A. Increase in leukocytes
B. Low calcium level
C. Platelet disorder
D. Deficiency of clotting factors
Answer: C

Example 2
Question:
Most common cause of acute cervicitis is :
A. E. Coli
B. Chlamydia
C. Pseudomonas
D. Gonococcus
Answer: D

Please answer the question below. Your response format should be a single
letter for the best answer option, that is A, B, C, or D.

Question:
Incubation period of HBV is -
A. 45 to 180 days
B. 6 to 60 days
C. 10 days
D. 10 hrs
Answer: A
```

Fig. 2. Example of train prompt with a set of instructions, 2 examples, the target question, and the correct answer to predict. Test prompts follow same template and set of instructions, except the correct answer to predict is excluded. In experiment, 10 examples were used for both train and test prompts.

Finally, I would like to note that most of the template code and hyper-parameters used for this fine-tuning task came from this great online tutorial [1]. Minor modifications were made, specially to load the model and make it work on a single (limited-resource) GPU. But, most of the effort was spent on preparing the training data, monitoring the fine-tuning job, and evaluating the fine-tuned model thereafter.

3.4 Testing Llama-based models

I tested Llama-2 7B chat (base), Llama-2 7B (fine-tuned), and Llama-3 8B (released on April, 2024). Each model was separately loaded onto a GPU (Google Colab) and invoked on all 4.2k test prompts (sequentially). On average, it took 30 seconds for each model to return a response. Thus, it took on average 35 continuous hours to generate (and save) all test prompt responses for each Llama model.

3.5 Testing GPT-3.5 Turbo

I also tested OpenAI's GPT-3.5 Turbo. However, unlike Llama-based models, GPT-3.5 Turbo is not open source; the only way to invoke it is via REST APIs. More specifically, I had to deploy the model on Azure OpenAI service and invoke it with OpenAI's Python SDK. The good news is a GPU was not required (I could run a script from my CPU). The bad news is due to network latency, on average it took GPT-3.5 Turbo 43 seconds to generate a response. To make matters worse, Azure OpenAI service rate limits requests, thus not allowing for significant parallelization. End-2-end, it took approximately 50 continuous hours to generate (and save) GPT-3.5 Turbo's responses for all 4.2k test prompts.

3.6 Processing and evaluating LLM responses

Once all test prompt responses were collected for each LLM, the final step was to process and evaluate the model's performance. Primary evaluation metrics were accuracy and weighted F1-score.

Because each LLM was given explicit (similar) instructions to respond with *A*, *B*, *C*, or *D*, I assumed the first non-empty character in LLM's response to be the predicted answer choice. However, some of the tested LLMs, particularly Llama-2 7B (base), did not respond accordingly. Thus, any unexpected or invalid answer choice was labeled as **none** and treated as incorrect. Finally, calculating evaluation metrics did not require any special compute resources. Ground truth (correct answer choices) were extracted from MedMCQA.

4 RESULTS

Figure 3 shows the distribution of predicted answer choices per model and in test dataset (ground truth distribution). Some key observations:

- Answer choices (ground truth) are not evenly distributed in the test set. *A* is the most common answer choice; 1347/4182, approximately 32.21%, the correct answer choice is *A*.
- Llama-2 7B chat (base) model performed the worst; for most questions it did not respond with *A*, *B*, *C*, nor *D*. The model's most common response was *"Please let me know your answer."* Thus, indicating Llama-2 base model did not understand the task at hand.
- Compared to the base model, fine-tuned Llama-2 7B performed much better. Fine-tuning helped the model understand the task at hand, that is to responded with a valid answer choice *A*, *B*, *C*, or *D*.
- Out of the box Llama-3 8B (base) model performed much better than either Llama-2 (base or fine-tuned) model. More specifically, without having the need to fine-tune, Llama-3 understood better the task at hand; it failed to respond with a valid answer choice only for 95 out of the 4.2k test prompts.
- All of GPT-3.5 Turbo's responses to 4.2k test prompts were valid answer choices.

Figure 4 shows the calculated accuracy and weighted F1-scores per model. Some key observations:

- Llama-2 7B chat (base) model performed the worst, with near zero percent accuracy. It only got one question correct.
- Fine-tuned Llama-2 model performed much better compared to base, with roughly 29.46% accuracy. However, a greedy algorithm that always predicts *A* would have achieved a much better accuracy of 32.21%.
- Out of the box Llama-3 8B (base) model performed much better than either Llama-2 (base or fine-tuned) model, with roughly 41.56% accuracy. Furthermore, Llama-3 also outperforms a greedy algorithm that always predicts *A*.
- GPT-3.5 Turbo outperformed all other models, with roughly 55.25% accuracy.

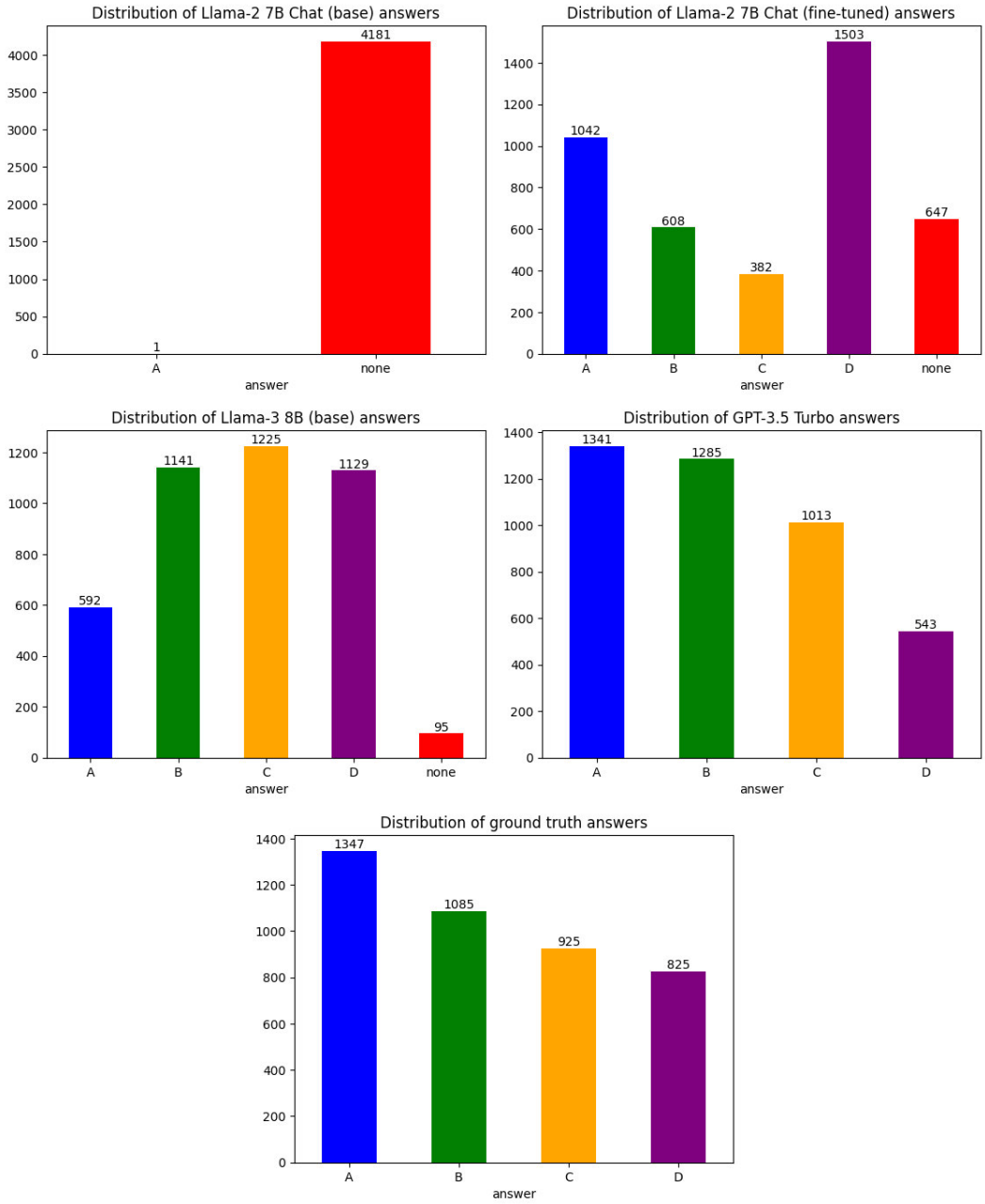

Fig. 3. Distribution of predicted answers across models and ground truth.

The high-level impacts of these results are:

- Fine-tuning open source Llama-2 base model helped improve its performance. However, improvement was not enough to outperform a greedy approach.
- Llama-3 8B and GPT-3.5 Turbo outperformed fine-tuned Llama-2 and greedy approach.

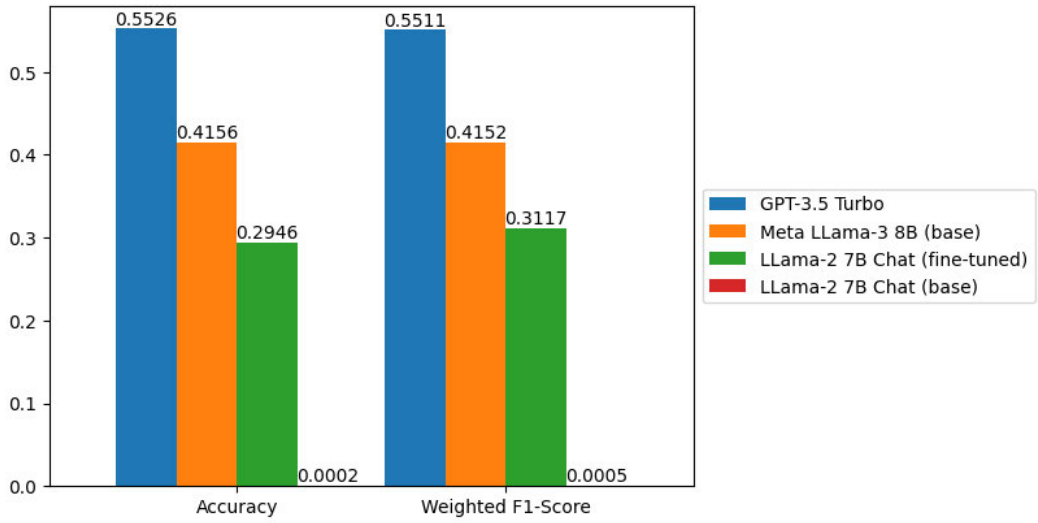

Fig. 4. Initial condition

5 CONCLUSION

In summary, for this project I managed to accomplish:

- Make use of public a multiple-choice dataset, i.e., MedMCQA, to benchmark performance of various large language models on medical Q&A.
- Make use of a GPU to fine-tune, serve, and improve the performance of open source Llama-2 base model for medical Q&A.

Future work to improve upon and expand on:

- Experiment with other open source LLMs, e.g., Minstral 7B, etc.
- Experiment with larger models, e.g., Llama-2 70B (parameters), Llama-3 70B, GPT-4, etc. Main limitations are:
  - Monetary cost of OpenAI requests. GPT-4 is more expensive than to GPT-3.5 Turbo.
  - Larger models require more and bigger GPUs. These are not cheap.
  - More time, i.e., days, is required to train and fine-tune larger models.
- Train and fine-tune with more data and different medical knowledge sources. The main limitation is lack of compute, time commitment, and high cost.
- Experiment with more prompt engineering techniques. Ideally, try to include external medical knowledge to assist the large language model in generating the correct answer.
- Explore methods of explainability to understand the reasoning behind an LLM's reasons in its answers. To be more specific, an LLM should be able to back and justify its response with factual evidence and logical reasoning.
- Finally, medical Q&A goes beyond simple multiple choice Q&A. Future work includes:
  - Support to answer free-form and open-ended medical questions.
  - Multi-modal support, e.g., ask question about an x-ray or MRI scan.
  - Evolve medical knowledge base and correct the model on a day-2-day basis.
  - Foresee and generate potential Q&A pairs, e.g., for a patient with specific set of symptoms, diagnosis, and lab results, provide a personalized FAQs.

REFERENCES

[1] Abid Ali Awan. 2023. Fine-Tuning LLaMA 2: A Step-by-Step Guide to Customizing the Large Language Model. https://www.datacamp.com/tutorial/fine-tuning-llama-2

[2] Hugging Face. [n. d.]. Methods and tools for efficient training on a single GPU. https://huggingface.co/docs/transformers/v4.40.1/perf_train_gpu_one

[3] Zhen Guo and Yining Hua. 2023. Continuous Training and Fine-tuning for Domain-Specific Language Models in Medical Question Answering. arXiv:2311.00204 [cs.CL]

[4] Valentin Liévin, Christoffer Egeberg Hother, Andreas Geert Motzfeldt, and Ole Winther. 2023. Can large language models reason about medical questions? arXiv:2207.08143 [cs.CL]

[5] Sourab Mangrulkar and Sayak Paul. 2023. PEFT: Parameter-Efficient Fine-Tuning of Billion-Scale Models on Low-Resource Hardware. https://huggingface.co/blog/peft

[6] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]